DSH-SUPERAGENT

The Autonomous Engineering System, Designed End to End

Evaluation of the prototype · the improvement roadmap · UI/UX design for the console, CLI and chat surfaces

BASED ON	github.com/ibrahim-hero/dsh-superagent-prototype (fork of deepseek-ai/dsh-superagent-prototype)
DOCUMENT	Product vision, capability evaluation, improvement roadmap and product UI/UX design
VERSION / DATE	v1.0 · November 11, 2026
STATUS	Proposal — combines analysis of the public prototype with a new product design
AUDIENCE	Engineering leadership, product design, and the dsh-superagent maintainers

DESIGN PRINCIPLE INHERITED FROM THE PROTOTYPE

"Take as little human attention as possible,
"and give as much value as possible."

— docs/000-philosophy.md, dsh-superagent

Prepared with OpenCode · Sources: public repository docs and source (docs/000–008, src/) and code.claude.com/docs

Contents

0 Executive Summary . . . 3

I Evaluation of the Prototype . . . 3

I.1 What the prototype actually is . . . 3

I.2 Genuine strengths (keep and build on these) . . . 3

I.3 Real gaps (where "more powerful than Claude Code" must be won) . . . 4

I.4 Scorecard . . . 4

II The Improvement Roadmap . . . 5

II.1 Evaluating the earlier plan — and what I would add . . . 5

II.2 The unified roadmap — twelve capabilities . . . 6

II.3 Success metrics (how we know it is more powerful) . . . 6

III UI/UX Design — the Product Surface . . . 8

III.1 Design principles . . . 8

III.2 Surfaces and information architecture . . . 8

III.3 CLI design . . . 8

III.4 Superagent Console (web) — screens . . . 9

III.5 Interaction patterns and components . . . 11

III.6 Visual design system . . . 11

III.7 Chat surface UX (GitHub / Feishu) . . . 12

III.8 Four key user flows . . . 12

III.9 UX metrics (design is measured too) . . . 12

IV Implementation Map — onto the Existing Codebase . . . 14

V Build Order and Milestones . . . 14

VI Target State vs Claude Code — Feature by Feature . . . 15

A Appendix A — Glossary (from docs/001, condensed) . . . 16

B Appendix B — Sources . . . 16

0 Executive Summary

dsh-superagent is a research prototype from the DeepSeek Harness team (publicly forked at github.com/ibrahim-hero/dsh-superagent-prototype) that runs a proactive coding agent for one GitHub repository. It listens to GitHub (and optionally Feishu), gates every incoming event through a versioned Policy, turns authorized requests into Tickets, executes each Ticket in its own worker Session and directory, and reports back through durable, ordered message queues. A web panel shows only what needs a human. Its stated philosophy — "take as little human attention as possible, and give as much value as possible" — is the right north star for an agent product.

This document does three things. **Part I** evaluates the prototype as it stands: what is genuinely strong (its domain model, durable effect queue, outbox, and recovery semantics are better than most agent harnesses) and where the real gaps are (readiness is self-reported, there is no persistent project knowledge, execution is not sandboxed, there is no benchmark, and the UI is a supervision panel rather than a workbench). **Part II** turns the earlier improvement discussion into a unified, prioritized roadmap of twelve capabilities, including new additions such as a benchmark/eval harness, cost and attention accounting, approval gates, and speculative execution. **Part III** is a complete UI/UX design for the product across its three surfaces — CLI, web console, and chat — with principles, information architecture, screen designs, interaction patterns, and a visual design system. **Parts IV–V** map the plan onto the existing codebase and sequence the build. **Part VI** benchmarks the target state against Claude Code feature by feature.

> **The one-sentence thesis**
>
> Beat Claude Code not by having a better model, but by being a better **system**: evidence-backed completion, durable checkpoints, persistent project knowledge, sandboxed verification, coordinated multi-agent work, and an interface that spends human attention only when it must — then measures that attention and reduces it over time.

I Evaluation of the Prototype

I.1 What the prototype actually is

The pipeline is clean and auditable: **Source → Trigger → Thread → Eval (gate + model assessment) → Decision → Ticket (with an immutable Mandate) → Assignment → Executor worker Session → Reports → Channel replies**. Work moves through a durable **Effects** queue (assessment, launch, delivery, relay, notice, reply) that only runs after the SQLite transaction commits, and every outgoing message gets an Outbox receipt with ordered, idempotent delivery. The Engine is the single writer of domain state and exposes role-scoped interfaces (Sources, Main Session, People, Settings, Executor), so authorization is enforced in code, not in prompt text. Workers run as separate DSH Sessions (`worker-<ticket>`) in per-Ticket directories, with background-job limits, worker slots, retry/recovery, and a restart path that reconciles state instead of replaying it.

Figure 1 — Runtime composition, condensed from docs/007-architecture.md.

I.2 Genuine strengths (keep and build on these)

Strength	Why it matters	Where it lives
Disciplined domain model	Trigger/Thread/Eval/Mandate/Ticket/Assignment/Subscription with explicit "rejected concepts"; no vague agent state.	docs/001, src/core/*

Strength	Why it matters	Where it lives
Durable side effects	Effects enqueue inside the transaction and run after commit; retries with backoff; health only after 3 consecutive failures.	src/app/effects.ts
Ordered, idempotent messaging	Outbox receipts, per-Ticket-per-conversation ordering, gone/withdrawn semantics; no duplicate posts after crashes.	src/app/outbox.ts
Honest recovery	Restart/retry reconcile worktree, git and GitHub before repeating external actions; read/unread Inputs tracked precisely.	docs/003, src/core/assignments.ts
Authorization in code, not prompts	Gate + Mandate revalidation; role-scoped Engine interfaces; prompt hierarchy ranks facts above instructions above outside text.	src/policy.ts, docs/004
Attention philosophy	Act only when asked or by written rule; speak only when a person must act; one question at a time; panel shows only what needs a person.	docs/000, docs/005
Extensible Sources	A cookbook turns any event stream into a Source kind (schema, scope, channel, policy, polling) with tests.	docs/008
Per-Ticket isolation	Own worker session, directory, branch, PR claiming, worker slots and concurrency with a human-message bypass slot.	src/executor/dsh.ts

I.3 Real gaps (where "more powerful than Claude Code" must be won)

Gap	Evidence in the repo	Consequence today
"Ready" is self-reported	orchestrator_report takes state=ready with a free-text summary; no structured proof (tests, artifacts, metrics) is required.	PRs can be marked ready without executable evidence; trust must be earned manually.
No persistent project knowledge	Sessions read AGENTS.md and resources on demand; nothing learns from ended Tickets.	Every Ticket rediscovers architecture, conventions and past mistakes.
Execution is not isolated	Workers run as the operator's OS user behind CLI shims; docs state this is "defaults, not a security boundary".	A worker can touch anything the operator can; parallel experiments are unsafe.
No benchmark or eval loop	Tests cover domain rules, not end-to-end agent quality; no trace-based scoring.	Improvements to prompts/policy cannot be measured; no regression net for the agent itself.
Single repository	One GitHub repo per deployment by design.	Cross-service features (API + client + infra) need manual coordination.
No cost / attention accounting	Health board tracks failures, not tokens, dollars, or human interruptions per Ticket.	The philosophy ("less attention") is unmeasured; noise can grow silently.
UI is supervision-only	Panel tabs: Overview/Source/Trigger/Eval/Execution/Settings — built to inspect, not to work in. No first-party CLI or IDE surface.	Daily use still happens in DSH chat, not in a purpose-built product.
Auto-actions are prompt-only	Auto-merge rules live in Delivery requirements text; no first-class approval gate with diff preview.	Merging/deploying needs a human in the loop by hand.
Single approach per Ticket	One worker per Ticket; subagents exist but there is no speculative parallel exploration.	Hard bugs get one hypothesis at a time.

I.4 Scorecard

Dimension	Score /10	Rationale
Domain model & authorization	9.0	Explicit, auditable, enforced in code; Mandates and Policy versions are exactly right.

Dimension	Score /10	Rationale
Durability & recovery	8.5	Effects + Outbox + restart reconciliation are production-grade; checkpoint granularity still coarse.
Execution depth / verification	4.0	Ready is a claim, not evidence; no reproduce→fix→verify loop enforced.
Project memory & knowledge	2.0	Nothing persists across Tickets beyond raw records.
Isolation & security	3.0	Shims and blocked commands only; runs as operator OS user by design.
Autonomy & learning	5.0	Observe mode records proactive candidates, but nothing learns from outcomes.
Observability	7.0	Health board, receipts, panel views — strong; but no traces, metrics, or evals.
Extensibility (Sources/plugins)	8.0	SourceKind protocol + cookbook is genuinely good.
Product surfaces (CLI/IDE/UI)	5.5	Panel is excellent for supervision; no workbench, no CLI of its own.
Multi-repo orchestration	1.0	Explicitly out of scope today.

Verdict

This prototype is the best **orchestration skeleton** we have seen in public: the hard part — reliable, auditable delegation with honest failure semantics — is already solved. What it lacks is everything that makes an agent **trustworthy at depth**: proof of work, memory, isolation, measurement, and a real product surface. Those are exactly the gaps the roadmap below closes.

II The Improvement Roadmap

II.1 Evaluating the earlier plan — and what I would add

The previous discussion proposed ten directions: evidence-backed readiness, durable checkpoints, project knowledge, explicit work contracts for subagents, hypothesis-driven debugging, adaptive model routing, validated skill learning, cross-repo workflows, reproducible sandboxes, and a better everyday interface. **All ten hold up against the code.** Three corrections and five additions make them actionable:

#	Addition / correction	Why
A1	**Build the benchmark + eval harness first (Phase 0), before any feature.**	Without trace-based scoring you cannot prove any later improvement; the philosophy demands measurement of attention too.
A2	**Add cost and attention accounting per Ticket** (tokens, USD, wall time, human interruptions, messages sent).	"Take as little human attention as possible" is currently a slogan; make it a metric the panel shows and the roadmap optimizes.
A3	**Promote approval gates to a first-class domain concept** (not just Attention questions): approve-with-diff-preview before merge/deploy/external side effects.	Extends the existing Attention mechanism; converts prompt-only auto-merge rules into auditable gates.
A4	**Speculative execution:** for hard/blocked Tickets, spawn N isolated hypotheses in parallel worktrees with cheap models first, then verify and merge the winner.	Turns debugging and risky refactors from single-threaded guessing into a tournament with evidence.
A5	**Decision replay / time-travel:** the event log (DomainEvents + receipts + prompt revisions) already exists — expose replay of any Ticket decision-by-decision.	Debugging the agent itself becomes possible; feeds the eval harness.
C1	**Correction:** sandboxes are not optional polish — make containerized execution a Phase-2 hard requirement, not an enhancement of the shim layer.	The docs themselves say shims are "not a security boundary"; a product that runs as your OS user cannot be trusted with auto-merge.

#	Addition / correction	Why
C2	**Correction:** knowledge must be **sourced and freshness-stamped** (file+commit, doc, or verified outcome), and invalidated when code changes.	Un-curated memory hallucinates; curated memory compounds.
C3	**Correction:** notification policy engine (per-channel rules, quiet hours, digest mode) as part of the Channel layer, not prompt text.	The outbox already serializes per conversation — add policy on top so noise is engineered down, not apologized for.

II.2 The unified roadmap — twelve capabilities

Capability	What it does	Impact	Effort	Phase
1. Benchmark & eval harness	Curated task set from real Tickets + synthetic repo tasks; trace-based scoring (did it verify? did it regress? how much attention?); runs in CI.	Critical	M	0
2. Cost & attention accounting	Per-Ticket ledger: tokens/USD/time/messages/human interruptions; panel widgets; alerts on budget breach.	High	S	0
3. Evidence-backed readiness	Verification service behind the Executor port: acceptance criteria → commands → exit codes → artifacts → before/after metrics; ready requires green evidence; re-verify on new commits.	Critical	L	1
4. Durable checkpoints	Structured per-Assignment checkpoint (plan, facts, verified state, external actions done, next step); recovery reconciles instead of re-asking the model.	High	M	1
5. Approval gates	First-class Approval domain object with diff/artifact preview, risk badge, expiry; auto-merge and deploys pass through it.	High	M	1
6. Sandboxed execution	Per-Ticket containers (or rootless runtimes) with scoped credentials, resource limits, snapshots; replaces shim-only guardrails.	Critical	L	2
7. Project knowledge index	FTS + symbol graph + embeddings over code/docs/decisions; every fact carries source commit and freshness; invalidated on change; fed by ended Tickets.	Critical	L	2
8. Coordinated work contracts	Subtask DAG with objectives, acceptance criteria, inputs/artifacts, budgets; overlap detection on files/APIs; integration step verifies the whole.	High	L	2
9. Adaptive routing & speculation	Per-task model/tool selection by measured success; escalation on stall; N parallel hypotheses in isolated sandboxes for blocked/hard Tickets.	High	L	3
10. Skill learning loop	Extract candidate lessons from successful Tickets; regression-test them on the benchmark; publish versioned skills only if they win.	High	M	3
11. Multi-repo orchestration	Workspace of N repos; cross-repo Tickets with dependency ordering (API → backend → client); shared knowledge graph.	High	L	3–4
12. Product surfaces	First-party CLI, redesigned web console (Part III), IDE extension, mobile approve; chat stays as a Source.	Critical	L	0→4

Impact: Critical = product-defining, High = major differentiator. Effort: S < 2 wks, M ≈ 2–4 wks, L ≈ 1–2 months (small team). Phase refers to Part V.

II.3 Success metrics (how we know it is more powerful)

Metric	Baseline (today)	12-month target
Verified completion rate on benchmark tasks (evidence-green endings)	not measured	≥ 80% on curated set, with 0 regressions shipped
Human interventions per Ticket (questions + stops + manual fixes)	not measured	median ≤ 1; p90 ≤ 3
Messages sent per completed Ticket (noise)	not measured	≤ 3 (ack + result + at most one question)
Cost per verified-completed task (USD, all models)	not measured	−60% vs Phase-0 baseline via routing
Time from request to reviewable PR (p50)	not measured	−50% vs Phase-0 baseline
Crash/restart recovery success (resume without human)	qualitative	≥ 95% of interrupted Assignments resume cleanly
Knowledge reuse: % of Tickets that read a fact learned by an earlier Ticket	0%	≥ 40%
Security: worker escapes / credential exposure in sandbox tests	n/a (no sandbox)	0 in red-team suite

III UI/UX Design — the Product Surface

The prototype ships a supervision panel inside DeepSeek Harness, but there is no first-party way to work with the agent day to day. This part designs the product surface: a CLI for speed, a web console ("Superagent Console") for supervision and review, IDE integration for diffs, and chat (GitHub/Feishu) kept as a source, not the interface. Everything below follows the prototype's own philosophy: every pixel must earn the attention it costs.

III.1 Design principles

Principle	Meaning in practice
P1 · Attention is currency	Default view answers "does anything need me now?" in under 2 seconds. Healthy states take zero space. Every notification must name the action it enables.
P2 · Evidence over adjectives	No "ready" without a green evidence badge. Status words are always backed by a receipt: command, exit code, artifact, metric.
P3 · One decision at a time	Questions and approvals arrive as a single structured card with a recommended answer — never buried in a report.
P4 · Progressive disclosure	Timeline first; internals (trigger ids, policy version, prompt revision, raw logs) one click down, always available for audit.
P5 · Keyboard-first, calm surface	Every action has a shortcut; Cmd+K palette everywhere; motion is subtle (120–200 ms) and respects reduced-motion.
P6 · Honest failure	Blocked/failed states show the last command, the error, and the exact next step — and are visually distinct from healthy states.

III.2 Surfaces and information architecture

Surface	Job to be done	Relationship
CLI (sg)	Start, watch, approve, ask, stop work from a terminal; scriptable; `--json` for automation.	Talks to the same Engine via the management API.
Superagent Console (web)	Supervise all Tickets, review evidence and diffs, approve gates, tune policy/prompts, explore knowledge.	Evolution of today's panel; same API, new design system.
IDE extension	Inline diff review of agent PRs, run/verify locally, "ask the agent" from a file.	Shares session state with console/CLI.
Chat sources (GitHub/Feishu)	Receive requests and deliver results where the team already talks.	A Source, not the product UI. Short messages, one question at a time.
Mobile approve	Approve/deny gates and answer questions from a phone in seconds.	Thin client over the approval API.

Figure 2 — One backend, four surfaces. The existing /orchestrator/api is already typed (src/app/api.ts), so surfaces are additive.

III.3 CLI design

A single binary `sg` with a small, memorable command tree. Streaming output by default (live status while a Ticket runs), `--json` for scripts, stable exit codes, and shell completions. The CLI never duplicates domain logic; it renders

Engine state.

Command	What it does
`sg run "fix the login timeout" [--repo o/r] [--mode observe|auto]`	Open a Ticket from the terminal (goes through the same Policy gate and Eval).
`sg ls [--stage working|needs-you|ended]`	List Tickets with stage chips, cost so far, and time-in-stage.
`sg watch <ticket>`	Stream the live timeline (trigger → eval → reports → evidence) until done.
`sg show <ticket>`	Full detail: mandate, policy version, inputs, assignments, evidence, receipts.
`sg diff <ticket>`	Open the PR diff locally with the AI summary and evidence badges inline.
`sg ask <ticket> "..."`	Send one structured message to the worker (counts as human attention — logged).
`sg approve <ticket>`	Approve a pending gate (merge/deploy/side-effect) with diff preview in the terminal.
`sg stop <ticket> [--reason]`	Withdraw the Mandate; work stops at the next step.
`sg retry <ticket>`	Retry a blocked/failed/delivery-failed Ticket (maps to Engine.retryTicket).
`sg panel`	Open the Console at the right page.
`sg bench [--set curated] [--json]`	Run the benchmark/eval harness and print the scorecard.
`sg costs [--today|--week]`	Attention and cost ledger: USD, tokens, messages, interruptions per Ticket.

```
$ sg run "checkout p95 is 4.2s — find the cause and fix it" --repo acme/shop
√ admitted (explicit, @mira) · policy v41 · eval 12s
» ticket tk_8f3a2  "checkout-latency"  worker tk_8f3a2 in work/checkout-latency
…
√ evidence green: p95 4.21s → 1.83s (k6, 5k VUs, commit 9c1e0ab) · tests 214 passed
► PR #482 opened · awaiting review · $1.94 · 41 min · 2 messages
$ sg approve tk_8f3a2        # when the gate is armed
```

Figure 3 — Happy path. Every line is a receipt-backed state, not a log soup. Exit codes: 0 ok, 1 user error, 2 system problem (mirrors the status script's contract).

III.4 Superagent Console (web) — screens

The console keeps the prototype's tab philosophy (Overview / Source / Trigger / Eval / Execution / Settings) but re-skins it into a workbench with a left rail, a top health strip, and an attention-first overview. Dark theme by default.

S1 · Overview — "does anything need me?"

Figure 4 — Overview wireframe. The needs-you rail is always first; everything else collapses when healthy (P1, P4).

S2 · Ticket / Run — the timeline

Figure 5 — Ticket timeline. Each node expands to raw detail (P4). Evidence nodes are clickable receipts (P2).

S3 · Verification / evidence drawer

Acceptance criterion (from the request)	Check	Result	Artifact
“p95 checkout latency below 2.5s”	k6 5k VUs, 10 min, commit 9c1e0ab	**PASS** 4.21s → 1.83s	grafana snapshot ↗
“no regression in order flow”	playwright e2e suite (38 tests)	**PASS** 38/38	ci run #5521 ↗
“unit tests pass”	pnpm test	**PASS** 214/214	ci run #5521 ↗
“no new authz surface”	semgrep + diff review	**PASS** 0 findings	report ↗

Figure 6 — Evidence drawer. A Ticket cannot be reported ready until every criterion has a green receipt (capability 3). New commits re-run the checks and flip the badge if they break.

S4 · Diff review with approval gate

PR view: file tree with change bars, an AI-written summary pinned on top, evidence badges next to the files they cover, inline review threads, and — when the policy arms it — a single **approval gate**: risk badge (size, blast radius, touched auth/payment files), the exact action it unlocks (“merge PR #482”), expiry, and Approve / Request changes / Stop. This is the Attention concept promoted to a first-class gate (capability 5).

S5 · Knowledge — searchable, sourced, fresh

One search box over the project knowledge index (capability 7). Results are fact cards: the fact, its source (file:line@commit, doc, or “verified in tk_…”), a freshness badge (green = still valid, amber = source changed, red = invalidated), and a mini graph of related facts. Example query: “if we change the auth middleware, what breaks?” → callers, tests, incidents, with links.

S6 · The question card (one decision at a time)

Figure 7 — Structured single question with a recommended answer (P3), routed to where the person asked.

S7 · Settings & policy timeline

Same groups as today (Sources / Eval / Execution / Prompts / Deployment) but with a **policy version timeline**: each version as a card showing what changed, which live Tickets were notified, and the benchmark delta before/after (capability 1 feeding the UI). Prompt overrides keep the existing “Default updated / keep mine” flow —

it is good and should be kept verbatim.

III.5 Interaction patterns and components

Pattern	Behavior
Command palette (Cmd+K)	Fuzzy over every action: tickets, sources, settings, “run benchmark”, “pause processing”. Keyboard-only path to everything (P5).
Approval gate modal	Diff summary + evidence badges + risk badge + one-line consequence + Approve/Request changes/Stop. Auto-focus Approve; Esc = dismiss (never approve). Mobile gets the same card as a push.
Attention card	The single pending human decision, pinned top of Overview and in the ticket header; answering clears it everywhere at once.
Pause switch	Global pause (exists today) surfaced as a top-bar toggle with a reason and auto-resume time; paused Tickets show “paused by you”.
Run controls	Watch (stream), Ask, Retry, Stop, Approve — always contextual to the Ticket’s stage; impossible actions are hidden, not disabled.
Streaming timeline	New events append live (SSE/WebSocket over the existing passes), with a “jump to latest” affordance when scrolled up.
Empty & error states	Empty = one sentence + one action. Errors show last command, exit code, and the retry button inline (P6).

Keyboard shortcuts

Key	Action	Key	Action
`Cmd+K`	Command palette	`g o`	Go to Overview
`Cmd+Shift+F`	Global search (triggers, threads, tickets, knowledge)	`a`	Approve pending gate
`w`	Watch current ticket	`s`	Stop ticket (confirm)
`d`	Open diff	`r`	Retry ticket
`?`	Glossary / help (domain terms, bilingual as today)	`p`	Pause/resume processing

III.6 Visual design system

Color tokens (semantic, not decorative)

Token	Value	Meaning
primary	#4D6BFE	Brand, links, primary actions
primary-dark	#3450D6	Headings, hover
ok / evidence-green	#1F9D6B	Healthy, verified, sent
attention	#E8A33D	Needs a person: questions, approvals, waiting
danger	#D9534F	Blocked, failed, dropped, errors
agent	#8B6FE8	Agent-originated content (bot messages, notices)
info	#2AA8A0	Progress, queued, informational
ink / muted / line	#0E1526 / #5B6478 / #D9DEEA	Text hierarchy and hairlines (light theme mirrors these)

Type, spacing, motion, accessibility

Aspect	Specification
Type	Segoe UI family (system-equivalent elsewhere): Display 34/light, H1 19/bold, H2 13.5/bold, body 9.5–10, mono = Consolas/SF Mono for code, ids, commands. Type scale 1.25.
Layout	4 px base grid; 18 mm page margins (print), 16/24 px gutters (web); 3-column max for dense tables; left rail 220 px fixed.
Radius & elevation	Radius 6 px (cards), 4 px (controls); elevation only for modals/palette (2 levels max — calm surface).
Motion	120–200 ms ease-out on state changes; streaming timeline appends without animation; prefers-reduced-motion honored globally.
Accessibility	WCAG AA contrast in both themes; status never conveyed by color alone (icon + label, as the panel's marks already do); full keyboard operability; focus rings visible.
Themes	Dark default (#0E1526 canvas), light theme mirroring the same tokens; agent messages always tinted agent-purple in chat contexts so humans and bots are never confused.

III.7 Chat surface UX (GitHub / Feishu)

Chat remains where requests arrive and results land, but the product makes those messages minimal and structured — this is the notification policy engine (addition C3) plus templates:

Moment	Message shape (≤ 1 sentence default, ≤ 100 words hard cap)
Acknowledgement	"Got it — I'll profile checkout and open a PR. (ticket tk_8f3a2)" — once, under the original message.
Question	One question + a link to the panel card; never a wall of text; names where it will be asked.
Result	"Fixed: p95 4.2s → 1.8s, tests green — PR #482." with link. Drafts are explicitly marked unfinished.
Blocked/failed	What failed, what was tried, the one decision needed. No "sorry" filler.
PR outcome	One final notice on merge/close (already exists as pullEndNotices — keep).

Policy controls: per-channel on/off, quiet hours, digest mode (batch non-urgent notices hourly), and a global "agent silence" switch. All messages carry the existing "automated agent" identity line.

III.8 Four key user flows

Flow	Steps (surface)
F1 · Request → verified PR	Mention @agent on GitHub (chat) → ack within ~1 min → Ticket appears on Overview (console/CLI) → watch timeline stream (CLI `sg watch` or console) → evidence drawer fills → PR with badges → human reviews diff (IDE/console) → merge (gate or manual).
F2 · Answer a mid-run question	Attention card pins on Overview + push/mobile + one chat message → operator picks the recommended option or types one line → answer lands in the original conversation → work resumes automatically.
F3 · Approve a gated action	Worker reports ready + policy arms gate → approval card with diff summary, risk badge, evidence → Approve from console, CLI (`sg approve`), or mobile → action executes with receipt → result posted.
F4 · Investigate the agent itself	Something looks wrong → open Ticket → timeline shows every decision with its policy version, prompt revision, model, and receipts → replay any decision (time-travel, addition A5) → file a benchmark case if it was a miss (`sg bench --add`).

III.9 UX metrics (design is measured too)

UX metric	Target
Time from opening console to knowing "does anything need me?"	< 2 s (needs-you rail above the fold)

UX metric	Target
Clicks/keystrokes to approve a gate	≤ 2 (Cmd+K → enter, or a → enter)
Time from question asked to answer delivered to worker	< 30 s when the person is online
Human attention events per completed Ticket	median ≤ 1 (tracked, not just asserted)
Accessibility	WCAG AA; 100% keyboard coverage of core flows

What makes this UI exceptional

It does not add screens to the prototype — it **makes the prototype's philosophy visible**. Attention-first overview, evidence badges instead of status adjectives, one-decision cards, sourced knowledge, and a measured noise budget turn "quiet by default" from a doc into the product's feel.

IV Implementation Map — onto the Existing Codebase

The prototype's dependency rules make this tractable: **src/core imports nothing and does no I/O**; ports live in `src/core/ports.ts`; app code implements them; the panel imports only types. Every roadmap capability slots in behind a port or a new module, without rewrites:

Capability	Where it lands	Key interfaces / files
Benchmark & eval harness	New top-level `bench/` + CI job; scores into Store	`bench/run.ts`, trace schema, `sg bench` CLI
Cost & attention ledger	New repo family in Store + health entry	`src/kv.ts` (cost:* keys), `src/infra/health.ts`, Sessions/Outbox counters
Evidence-backed readiness	New Evidence port beside Executor; report schema extended	`src/evidence/*`; `orchestrator_report` gains evidence refs; stage table gains evidence states
Durable checkpoints	Checkpoint records per Assignment; recovery reconciles	Store tickets/assignments + `src/core/assignments.ts` RECOVERY_NOTE flow
Approval gates	New domain object Approval + engine command; UI gate	`src/core/*` (alongside Attention), panel S4, mobile push
Sandboxed execution	Executor port gains sandbox runtime (containers); shims become defense-in-depth	`src/executor/*`, `src/infra/environment.ts` kept as inner layer
Knowledge index	New module; ingestion hook on ticket.ended; freshness invalidation on poll of changed files	`src/knowledge/*`, SQLite FTS5 + symbol graph, panel S5
Work contracts / DAG	Assignment gains subtask graph; overlap detection on changed paths	`src/core/assignments.ts`, `src/executor/dsh.ts` WorkerSlots
Adaptive routing & speculation	Role-model resolution becomes task-level; speculation manager spawns N sandboxes	`src/app/role-models.ts` → router; new `src/app/speculation.ts`
Skill learning loop	Extraction from ended Tickets → candidate skills → bench regression → publish	`src/app/prompts.ts` versions + skills registry + bench
Multi-repo	Deployment resolves N repos; Tickets gain workspace scope	`src/deployment.ts`, Engine scope fields
Product surfaces	CLI binary; console redesign; IDE extension	new `bin/sg` (tsdown entry), `src/client/*`, extension package

V Build Order and Milestones

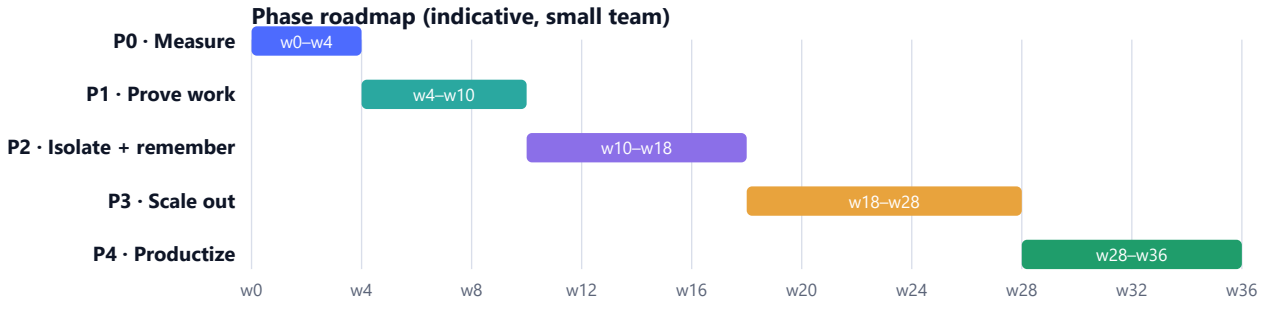


Figure 8 — Indicative phase plan for a small team.

Phase	Goal	Exit criteria
P0 · Measure (w1–4)	Benchmark + eval harness; cost/attention ledger; sg CLI v0 (ls/show/watch/ask/stop); console reskin of Overview.	Bench runs in CI with a scorecard; every Ticket shows cost; CLI covers the 5 core commands.
P1 · Prove work (w5–10)	Evidence-backed readiness; durable checkpoints; approval gates v1 (merge).	0 ready-reports without green evidence on the bench set; 95% clean resumes after kill -9; gates auditable.
P2 · Isolate + remember (w11–18)	Container sandboxes; knowledge index with freshness; notification policy engine.	Red-team suite: 0 escapes; ≥ 40% knowledge reuse on bench; messages/ticket ≤ 3.
P3 · Scale out (w19–28)	Work-contract DAG + integration verification; adaptive routing; speculative execution; skill learning loop; multi-repo alpha.	Bench verified-completion ≥ 80% with 0 regressions; routing cuts cost ≥ 60%; 2-repo feature shipped end-to-end by the agent.
P4 · Productize (w29–36)	Console full design (Part III); IDE extension; mobile approve; plugin/skill marketplace; air-gapped mode.	UX metrics in III.9 hit; external team runs it on their own repo with the SETUP flow.

VI Target State vs Claude Code — Feature by Feature

Capability	dsh-superagent today	Claude Code (code.claude.com/docs)	Target state (this plan)
Surfaces	Panel inside DSH + chat	Terminal, IDE, desktop, web, mobile, remote control	sg CLI + Console + IDE + mobile approve + chat sources
Memory	AGENTS.md per session; raw records	CLAUDE.md + auto memory across sessions	Sourced, freshness-stamped knowledge graph + learned skills
Verification	Self-reported ready	Runs commands; user judges	Evidence receipts gate readiness; auto re-verify on new commits
Long-running work	Worker slots, 30-min background limit, recovery note	Background agents, cloud sessions, routines/schedules	Durable checkpoints + sandboxed long runs + scheduled Tickets
Parallelism	Subagents under standard preset	Parallel sub-agents, dynamic workflows	Contract-based DAG + speculative hypothesis tournaments
Automation triggers	GitHub/Feishu Sources + admission table	GitHub Actions, Slack, channels, webhooks, schedules	Same Sources + webhook Source + routines; approval gates for side effects
Tools	gh/lark-cli shims, blocked commands	MCP servers, Chrome, broad toolset	MCP/plugin marketplace + sandboxed arbitrary tools
Safety	Prompt hierarchy + gate; shims only	Permissions, hooks, plan review	Mandates + gates + containers + red-team suite + scoped credentials
Team orchestration	One repo, one agent account, panel	Projects coordinating parallel sessions	Multi-repo workspaces, cross-repo Tickets, attention ledger
Self-improvement	Observe mode records candidates	Skills + hooks authored by humans	Benchmark-driven skill learning loop + decision replay
Cost/attention visibility	Health board only	Usage shown in app	Per-Ticket USD/attention ledger, panel + CLI, optimized as a metric

Bottom line

Claude Code wins today on surfaces, ecosystem and polish. The prototype wins on orchestration integrity. The plan closes the surface gap with the design in Part III and extends the integrity lead into proof, memory, isolation and learning — which is where an agent earns the right to act with less supervision than any human teammate needs.

A Appendix A — Glossary (from docs/001, condensed)

Term	Meaning
Source / Trigger / Thread	An event stream / one immutable event / ordered Triggers; the Eval unit.
Eval → Decision	Gate (deterministic) + model assessment → accept / stop / ignore / unavailable.
Mandate	Immutable grant of what an accept allows: strength, scope, policy version.
Ticket / Assignment	A unit of authorized work / its binding to one Executor instance.
Subscription / fan-out	Ticket↔Thread relation; matching Triggers reach the Ticket as context.
Report / Attention	Worker status (progress…dropped) / one structured question waiting for a person.
Effects / Outbox	Durable post-commit side-effect queue / ordered idempotent message receipts.
(new) Evidence / Checkpoint / Gate	Verification receipts behind ready / resumable structured state / approval object for side effects.

B Appendix B — Sources

- github.com/ibrahim-hero/dsh-superagent-prototype — README, SETUP.md, docs/000–008 (philosophy, domain, authorization, execution, prompts, panel, deployment, architecture, cookbook), package.json, src/core/*, src/app/*, src/executor/*, src/sources/*, src/infra/environment.ts, prompts/worker.md (fetched 2026-11-11).
- code.claude.com/docs — Claude Code overview and feature set (terminal/IDE/desktop/web, memory, MCP, sub-agents, hooks, routines, remote control, channels, GitHub Actions) for the Part VI comparison.

All prototype quotations (e.g. “defaults, not a security boundary”, stage tables, effect kinds) are from the public repository files listed above.